

 Serviço Nacional de Aprendizagem Industrial Santa Catarina	AVALIAÇÃO PRÁTICA		Desempenho
	Data:		
	Docente: <i>Carlos Roberto da Silva Filho</i>		
	Curso Técnico em <i>Desenvolvimento de Sistemas</i>		
	Unidade Curricular: <i>Programação de Aplicativos</i>		
	Turma: <i>3B</i>		
	Estudante:		

CAPACIDADES
H2 - Instalar ferramentas de acordo com requisitos de hardware, software e parâmetro de configuração H4 - Integrar banco de dados por meio da linguagem de programação H5 - Aplicar métodos e técnicas de programação H10 - Utilizar padrão de projeto para desenvolvimento de aplicativos H11 - Utilizar técnicas de integração de aplicações com banco de dados na estruturação do sistema H12 - Utilizar frameworks para o desenvolvimento de aplicativos H13 - Reconhecer especificações técnicas e paradigmas de linguagem de programação

CONTEXTUALIZAÇÃO
Uma grande rede varejista nacional está expandindo suas operações e iniciando o planejamento estratégico para o lançamento de sua plataforma oficial de e-commerce. Atualmente, os processos de controle de estoque, vendas presenciais e cadastro de clientes são gerenciados de forma isolada em cada unidade física. Para unificar a operação e validar a infraestrutura tecnológica que apoiará o sistema definitivo, a diretoria de tecnologia da empresa decidiu investir no desenvolvimento de um Protótipo Piloto de Backoffice chamado de Sistema de Compras Interno do tipo web. O objetivo principal deste piloto é validar a arquitetura de software proposta, garantindo o fluxo contínuo e íntegro dos dados: desde a integração com catálogos externos para a carga inicial de produtos, passando pela gestão transacional de usuários e movimentações de estoque, até a consolidação de relatórios analíticos e gráficos para a tomada de decisões gerenciais. Como a empresa está reestruturando seu departamento de TI e ainda não possui uma equipe interna, a diretoria contratou um Product Owner (P.O.) especializado para mapear as regras de negócio, definir as restrições técnicas e acompanhar de perto o desenvolvimento desta 1ª versão funcional do sistema. Você foi o desenvolvedor selecionado para participar deste processo de especificação e implementação. Caso este protótipo atenda com sucesso a todos os requisitos de negócio, arquitetura REST, critérios visuais de interface e relatórios analíticos e gráficos, sua solução será homologada como a base oficial do futuro sistema de compras interno do tipo web da empresa.

DESAFIO
Como desafio proposto pelo Product Owner (P.O.), você deverá projetar, especificar e implementar a versão piloto do Sistema de Compras Interno do tipo web, utilizando uma arquitetura full-stack baseada em Node.js no Back-End e HTML, CSS e JavaScript no Front-End (Padrão REST), utilizando o ORM Sequelize com MySQL. O banco de dados do projeto deverá se chamar "db_compras". Ficou estabelecido que o protótipo será composto por um sistema de navegação sempre acessível no topo (Navbar) que integra as seguintes telas: Tela Inicial: Informações do sistema de compras interno. Tela de Usuários: Interface para o gerenciamento (CRUD) dos dados dos usuários. Tela de Produtos: Interface para o gerenciamento (CRUD) do catálogo de produtos. Tela de Movimentação/Vendas: Interface para registrar as operações de compra registrando a entrada e saída dos produtos (histórico de movimentação completo). Tela de Relatório Analítico: Esta interface será responsável por exibir relatórios estratégicos na forma de Tabelas, com dados extraídos diretamente de Views, sendo os seguintes relatórios: de Produtos Críticos e de Volume Financeiro Comprado por Produto. Tela de relatório Gráfico: Esta interface apresentará ao gestor do sistema dois gráficos dinâmicos utilizando a biblioteca Chart.js, alimentados pelas rotas do backend conectadas às tabelas e views, que são: Estoque Físico Atual e Volume Financeiro de Compras. Tela de Dashboard dos produtos: Painel centralizador contendo os produtos na forma de "card".

Estrutura do Banco de Dados

1. Tabela **usuarios**

Contém os seguintes campos:

- codUsuario (id)
- Nome (firstName)
- Sobrenome (lastName)
- Idade (age)
- E-mail (email)
- Telefone (phone)
- Endereço (address)
- Cidade (city)
- Estado (state)

2. Tabela **produtos**

Contém os seguintes campos:

- codProduto (id)
- Nome (title)
- Descrição (description)
- Categoria (category)
- Preço (price)
- Percentual de desconto (discountPercentage)
- Quantidade (stock)
- Marca (brand)
- Imagem (thumbnail) - URL da imagem

3. Tabela **compras**

Responsável por registrar cada compra realizada por usuários, com os seguintes campos:

- ID da compra (chave primária)
- ID do usuário (chave estrangeira)
- ID do produto (chave estrangeira)
- Tipo Movimento (ENTRADA, SAIDA)
- Quantidade Movimentada
- Preço unitário
- Desconto aplicado (%)
- Preço final (calculado)
- Forma de pagamento (DEBITO, CREDITO, DINHEIRO)
- Status da Compra (PAGA, PENDENTE)
- Data da compra

Carga Inicial em Lote (Bulk Create)

Para validar a capacidade de integração do sistema, o desenvolvedor deverá criar um mecanismo que consuma dados de duas APIs públicas através do comando FETCH.

- Produtos: <https://dummyjson.com/products>
- Usuários: <https://dummyjson.com/users>

Estes dados externos devem ser tratados no backend para inserção nas tabelas locais utilizando a operação "bulkCreate" do Sequelize.

Dashboard, Gráficos e Relatórios (Views)

O Dashboard, relatórios e gráficos serão o coração analítico do sistema e deverão exibir os dados de forma dinâmica. Os gráficos serão gerados com Chart.js - <https://www.chartjs.org/>.

- 1 - Relatório de Produtos Críticos (vw_produtos_criticos): Voltado para identificar o gargalo de estoque do varejista. A tabela deve exibir: codigo_produto (mapeado de codProduto), nome (mapeado de title), categoria e quantidade atual.
- Regra de Negócio do relatório 1: A View deve filtrar e retornar apenas os produtos cujo estoque atual seja inferior a 10 unidades.
- 2 - Relatório de Volume Comprado por Produto (vw_volume_compras): Voltado para exibir o histórico de movimentações financeiras das saídas do estoque. A tabela deve exibir: nome (do produto), quantidade total movimentada e valor financeiro movimentado.
- Regra de Negócio do relatório 2: O valor financeiro movimentado deve ser calculado diretamente na View multiplicando a quantidade total de itens comprados (saídas) pelo preço unitário registrado no momento da compra (Quantidade Movimentada x Preço unitário).
- Relatório Gráfico: Esta interface apresentará ao gestor do sistema dois gráficos dinâmicos utilizando a biblioteca Chart.js, alimentados pelas rotas do backend.
- Gráfico 1 (Estoque Físico Atual): Gráfico do tipo Barras Verticais, que exibe o saldo atual de produtos em depósito. Eixo Horizontal (X): Nome do produto (title) e Eixo Vertical (Y): Quantidade de itens em estoque (stock).
- Regra de Negócio do gráfico: A View deve filtrar e retornar apenas os produtos cujo estoque atual seja inferior a 10 unidades.
- Gráfico 2 (Volume Financeiro de Compras): Gráfico do tipo Barras Horizontais, focado no faturamento e saída de capital por mercadoria. Ele consumirá os dados da view de movimentações (vw_volume_compras). Eixo Horizontal (X): Valor financeiro total movimentado. Eixo Vertical (Y): Nome do produto.
- Regra de Negócio do Gráfico 2: Por limitações de layout, este gráfico deve exibir estritamente os 5 produtos com o maior valor financeiro movimentado.

Testes de Integração (REST Client)

Independente da interface gráfica, o backend deverá conter na raiz do projeto um arquivo de testes automatizados chamado teste.http, configurado para a extensão REST Client do VS Code. O script de testes deve conter blocos separados (por ####) capazes de testar e demonstrar *in loco*: a saúde do servidor, o gatilho da carga em lote por bulkCreate, o CRUD completo das entidades e os cenários de movimentação de estoque com sucesso e com erro por falta de saldo, relatórios e gráficos.

Entregáveis da Especificação Técnica

Para homologação do projeto junto ao P.O., deverão ser gerados e entregues os seguintes artefatos:

- Diagrama de Caso de Uso geral em UML;
- Diagrama de Classes UML com os respectivos atributos e relacionamentos;
- Diagrama Lógico do Banco de Dados (Engenharia Reversa);
- O banco físico em backup no formato de um arquivo “.sql” apresentando as operações realizadas no banco de dados, assim como os dados que foram inseridos nas tabelas e views.
- Lista de requisitos funcionais, não funcionais e regras de negócio do sistema.
- Lista de Infraestrutura do sistema (software e hardware)
- Código-fonte completo, funcional, versionado no GitHub (mínimo de 5 commits) e comentado.

RESULTADOS E ENTREGAS

A pontuação só será validada se a funcionalidade estiver completa no front e back end.

As imagens dos diagramas devem ser no formato “.png”.

Os arquivos de texto devem ser no formato “.pdf”.

Todos os arquivos, pastas e códigos devem ser compactados no formato zip **sem** o **node_modules**.